

Week 42: From Backpropagation to Differential Equations

The network of week 41 once more – activations, output layers, the algorithm – and then the same network solving ordinary and partial differential equations, ending with physics-informed neural networks

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 42

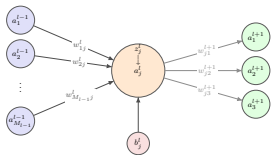
Monday lecture (2×45 min):

- ▶ **First 30 min, the reminder from last week:** the forward pass, the four equations of backpropagation and how to set the algorithm up; the activation functions and why there must be one; the output activation for regression (linear), binary classification (sigmoid) and K classes (softmax), each paired with its cost – with the code of week 41 run on the Runge function and on MNIST (Sections 8.6, 8.8.4–8.8.5, 8.11, 8.13)
- ▶ **Differential equations without data (Chapter 9):** the trial solution and the residual as a cost, derivatives of the network with respect to its *input*, and three ordinary differential equations – exponential decay, logistic growth, the Poisson equation – checked against the exact solution, forward Euler and finite differences (Sections 9.1–9.5)
- ▶ **A partial differential equation:** the diffusion equation in one dimension with a trial solution that satisfies all conditions exactly (Sections 9.6–9.7)
- ▶ **Physics-informed neural networks:** the conditions as soft constraints, what that costs in accuracy, and what it buys – an unknown diffusion coefficient recovered from forty noisy measurements (Section 9.10)

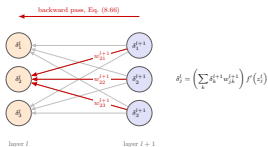
Tuesday (flipped classroom): your questions, then *one* case: the backward pass of this week's exercises – shapes, the feed-forward repetition, the four equations in code (**deadline Friday October 16**).

Reading: Chapter 9, Sections 9.1–9.7 and 9.9–9.10; Sections 8.6, 8.8 and 8.11 as a reminder. Goodfellow et al. Chapter 6; Raschka et al. Chapter 11; Lagaris, Likas and Fotiadis (1998); Raissi, Perdikaris and Karniadakis (2019).

Where we were: the forward pass and the four equations (8.5, 8.8.4)



$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \quad a_j^l = f(z_j^l)$$



Book Figures 8.6 and 8.13: the same graph, arrows reversed.

Forward, Eq. (8.21), samples in the rows of $A^0 = X$, W^l of shape (M_{l-1}, M_l) , the *output* activation in the last layer, every Z^l and A^l stored: $Z^l = A^{l-1}W^l + b^l$, $A^l = f(Z^l)$, $l = 1, \dots, L$.

Backward: the four equations (8.65)–(8.68), for a batch

$$\Delta^L = f'(Z^L) \circ \partial C / \partial A^L$$

start; $A^L = Y$ for the natural pairings (8.69)

$$\Delta^l = (\Delta^{l+1} (W^{l+1})^T) \circ f'(Z^l)$$

carry back through W^T , modulate by f'

$$\partial C / \partial b^l = \sum_{\text{samples}} \Delta^l$$

bias: the error itself

$$\partial C / \partial W^l = (A^{l-1})^T \Delta^l$$

weight: error $\times$ the entering signal

The rule. $\delta_j^l = \partial C / \partial z_j^l$ is the error of a node; the gradient of a weight is the error at the node it arrives at times the signal at the node it leaves. Two transposes in two different places: $(W^{l+1})^T$ sums over the *next* layer, $(A^{l-1})^T$ over the *samples*. The whole gradient costs about two forward passes, whatever the number of parameters (Proposition 4.2) – and a hand-written backward pass is always checked against finite differences or `jax.grad`.

Setting the algorithm up, and the code it became (8.8.5, 8.13)

Five decisions before the first iteration

1. The architecture: `sizes = [784, 100, 10]`.
2. The hidden activation f – and its derivative f' , which is what (8.66) uses.
3. The output activation, dictated by the *range* of the target, and the cost, dictated by its *distribution* (Section 3.6); penalties and their λ .
4. The optimiser (week 38) and its learning rate γ , the minibatch size M , the number of epochs.
5. The initialisation: variance $1/M_{l-1}$ or $2/M_{l-1}$ (8.73–8.74), never all zeros, never $\mathcal{N}(0, 1)$ with 784 inputs.

Each iteration, on a minibatch

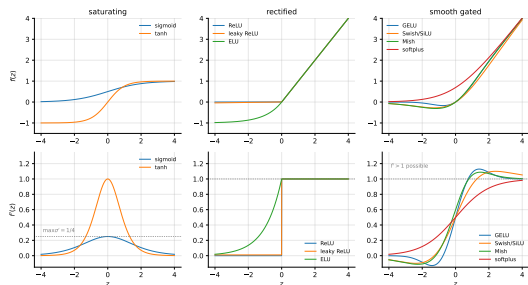
forward pass storing $Z^l, A^l \rightarrow$ output error $\rightarrow$
backward pass $\rightarrow$ gradients $\rightarrow$ update
 $w \leftarrow w - \gamma \partial C / \partial w$ (8.70). Then *check the gradient*:
finite differences (4.102) or `jax.grad`.

```
def feed_forward(X, layers, hidden_activation="sigmoid", head="mse"):
    f, _ = ACTIVATIONS[hidden_activation]
    a, z = [X], []
    for l, (W, b) in enumerate(layers):
        z1 = a[-1] @ W + b # Eq. (8.21)
        z.append(z1)
        a.append(OUTPUT[head](z1) if l == len(layers) - 1 else f(z1))
    return z, a

def backpropagation(X, Y, layers, hidden_activation="sigmoid", head="mse", lmbd2
                    =0.0, lmbd1=0.0):
    _, fp = ACTIVATIONS[hidden_activation]
    n = X.shape[0]
    z, a = feed_forward(X, layers, hidden_activation, head)
    delta = (a[-1] - Y)/n # Eq. (8.69), all three heads
    grads = [None]*len(layers)
    for l in range(len(layers) - 1, -1, -1):
        W, _ = layers[l]
        dW = a[l].T @ delta + 2.0*lmbd2*W + lmbd1*np.sign(W) # (8.68) + penalties
        db = np.sum(delta, axis=0) # (8.67)
        grads[l] = (dW, db)
        if l > 0:
            delta = (delta @ W.T) * fp(z[l - 1]) # (8.66)
    return grads
```

ACTIVATIONS holds (f, f') pairs, OUTPUT the three heads identity, sigmoid, softmax; train loops over epochs and minibatches. Today we add *one* row to ACTIVATIONS: "tanh": (tanh, tanh_prime). Part 2 says why.

Activation functions: what matters is the derivative (Section 8.6, Figure 8.9)



Nine activations in three families (top) and their derivatives (bottom); dotted lines at $\max \sigma' = \frac{1}{4}$ and at $f' = 1$.

Saturating (8.22): $\sigma(z) = 1/(1 + e^{-z})$, $\sigma' = \sigma(1 - \sigma) \leq \frac{1}{4}$; $\tanh(z) = 2\sigma(2z) - 1$, $\tanh' = 1 - \tanh^2 \leq 1$, centred on zero – always the better of the two in a hidden layer. Both have $f' \approx 0$ for $|z|$ large: a saturated unit stops learning, and L sigmoid layers multiply the error by at most 4^{-L} (8.71).

Rectified (8.23)–(8.25): $\text{ReLU}(z) = \max(0, z)$, $f' = 1$ on the active side – the gradient passes undiminished, half the units are exactly zero; *dying ReLU* when the input is negative for every sample. Leaky ReLU, ELU repair the dead side.

Smooth gated (8.26)–(8.28): GELU $z\Phi(z)$, Swish $z\sigma(z)$ – ReLU without the kink.

Two rules, and a third for today

(i) The hidden activation must be non-linear – Section 8.4: *not a polynomial* – or the network is one linear map. (ii) In the backward pass only f' appears, so pick one whose derivative does not vanish where the units live. (iii) When the *output* must be differentiated with respect to the *input*, f'' appears too – and for the whole rectified family $f'' \equiv 0$. Hence tanh in Part 2.

Why a hidden layer with a non-linear activation fits a non-linear function (Section 8.4)

The theorem, in its sharp form

A network with one hidden layer and a linear output is a universal approximator – any continuous F on a compact set to any accuracy ϵ (8.3) – *if and only if* the hidden activation is not a polynomial (Leshno et al. 1993). Cybenko proved it for sigmoids, Hornik for anything bounded and non-constant, and in mean square (8.5) rather than uniformly, which is the statement a statistician wants.

What it does not say

Nothing about *how many* units, nothing about whether gradient descent *finds* the weights: representability is not learnability (Section 8.4.4). Polynomials, Fourier series and splines are universal too; what distinguishes networks is depth – Section 8.4.3 shows a function a deep network represents with a polynomial number of units that a shallow one needs exponentially many for.

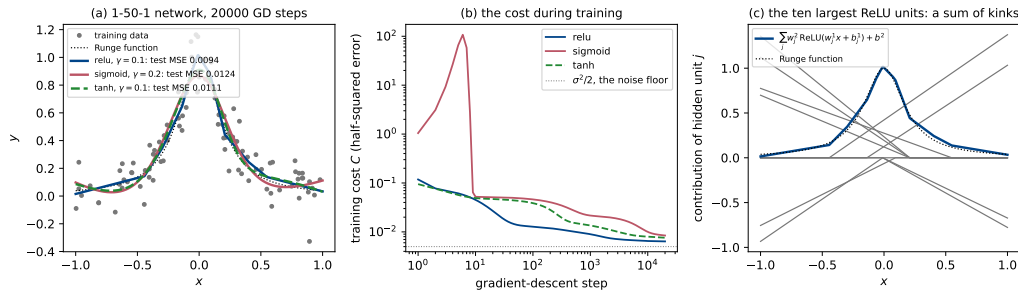
A constructive version for ReLU in one dimension (8.6)–(8.8)

$g(x) = c + \sum_{i=1}^N v_i \text{ReLU}(w_i x + b_i)$ is continuous and piecewise linear with at most $N + 1$ pieces, the kinks at $-b_i/w_i$ (Lemma 8.1) – and conversely every such function is a one-hidden-layer ReLU network. Put the kinks at $N + 1$ equally spaced knots and choose the v_i as the *changes of slope* of the chords, Eq. (8.8): the network is the piecewise-linear interpolant, and for an L -Lipschitz F on $[a, b]$

$$\sup_x |F(x) - g(x)| \leq \frac{L(b-a)}{2N} \quad (8.7).$$

A hidden unit is a kink, the output layer adds kinks with weights, and the non-linearity is what lets a sum of them bend; with sigmoids each unit is a smooth step.

Example A: the Runge function with ReLU, sigmoid and tanh hidden units (week42_runge.pdf)



The data of Project 1 ($n = 100$, $\sigma = 0.1$, seed 2026, a separate test set), the 1–50–1 network of week 41 with the linear output and the half-squared error, plain gradient descent, 20 000 steps. **(a)** Test MSE 0.0094 (ReLU, $\gamma = 0.1$), 0.0124 (sigmoid, $\gamma = 0.2$), 0.0111 (tanh, $\gamma = 0.1$); the noise variance is 0.01, so all three have found the function. **(b)** ReLU is at the noise floor after a few hundred steps; the sigmoid network first climbs to $C = 30$ and needs 10^4 steps; tanh is in between – the derivative up to 1 instead of $\frac{1}{4}$ and the centred output. **(c)** The ten largest of the fifty ReLU units, each a kink $w_j^2 \text{ReLU}(w_j^1 x + b_j^1)$; all fifty kinks lie inside $[-1, 1]$, and their sum plus b^2 (blue) is the network: a piecewise-linear function that bends where the Runge function does – Lemma 8.1 after training rather than by construction.

Pause and think 1: how many ReLU units does the Runge function need?

Question

Theorem 8.2 guarantees $\sup |F - g| \leq L(b - a)/(2N)$ for the interpolating network with N units. For $F(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ the steepest slope is $L = \max |F'| \approx 3.25$ (at $x = \pm 0.115$). How many units does the bound require for an error of 0.01, and how many did gradient descent use to reach the noise floor? Why the difference – and what would the *same* network with the identity as hidden activation have fitted?

Pause and think 1: how many ReLU units does the Runge function need?

Question

Theorem 8.2 guarantees $\sup |F - g| \leq L(b - a)/(2N)$ for the interpolating network with N units. For $F(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ the steepest slope is $L = \max |F'| \approx 3.25$ (at $x = \pm 0.115$). How many units does the bound require for an error of 0.01, and how many did gradient descent use to reach the noise floor? Why the difference – and what would the *same* network with the identity as hidden activation have fitted?

Answer

$N \geq L(b - a)/(2 \cdot 0.01) = 3.25 \cdot 2/0.02 = 325$ units for the guarantee; the trained network used 50 (and week 41's ℓ_1 run kept only 10 of them). The bound is a worst case for *equally spaced* kinks and *any* L -Lipschitz function; training puts the kinks where the function bends and leaves the flat tails to one or two units each, so the pieces are used far more economically – Figure (c). That is the gap between a representability bound and what an optimiser finds, in the favourable direction for once. With the identity as hidden activation the network is $g(x) = w^{2T}(w^1x + b^1) + b^2 = \alpha x + \beta$: a straight line, fifty-one parameters for a two-parameter model, and the best fit to the Runge data is the horizontal line at the mean, MSE 0.10. No kinks without a non-linearity; Section 8.4 made it an if-and-only-if.

Output activations: three tasks, three heads, one output error (Sections 8.11, 5.3, 5.6)

The output activation is dictated by the *range* of the target, the cost by its *distribution*; for the natural pairings the backward pass starts with prediction minus target, Eq. (8.69).

Task	Output activation	Loss L (one sample)	Likelihood of	δ^L
regression, $y \in \mathbb{R}$	linear, $a^L = z^L$	$\frac{1}{2}(a^L - y)^2$, (8.32)	a Gaussian (3.6)	$a^L - y$
binary, $y \in \{0, 1\}$	sigmoid, $a^L = \sigma(z^L)$	$-y \ln a^L - (1 - y) \ln(1 - a^L)$, (5.13)	a Bernoulli	$a^L - y$
K classes, y one-hot	softmax (8.75), $a_k^L = e^{z_k^L} / \sum_l e^{z_l^L}$	$-\sum_k y_k \ln a_k^L = -\ln a_{k^*}^L$, (5.28)	a categorical	$a^L - y$

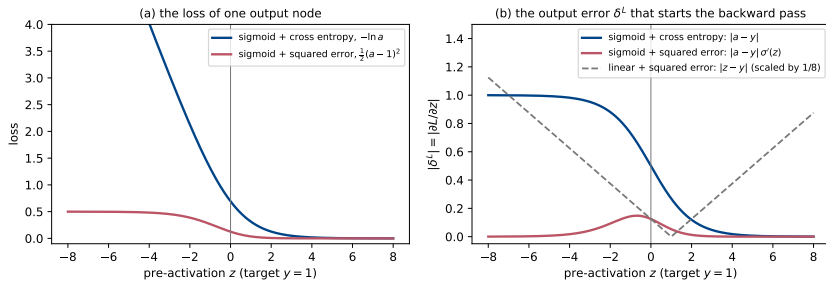
The two cancellations

Sigmoid: $\partial L / \partial a = (a - y) / [a(1 - a)]$ and $\sigma' = a(1 - a)$ – the σ' that shrinks every squared-error gradient is divided out. Softmax: the Jacobian $\partial a_k / \partial z_j = a_k(\delta_{kj} - a_j)$ is not diagonal, but $\delta_j^L = \sum_k (-y_k / a_k) a_k(\delta_{kj} - a_j) = -y_j + a_j \sum_k y_k = a_j - y_j$. The a_k cancels the $1/a_k$ of the logarithm, $\sum_k y_k = 1$ does the rest. Linear + squared error: $f' = 1$, nothing to cancel.

What the heads *mean*

A linear output can be 7; a sigmoid output is a probability $p(y = 1 \mid x)$; a softmax output is a *distribution* over K classes, positive and summing to one, computed from scores z_k that are only defined up to a common shift (subtract $\max_k z_k$ before exponentiating, Eq. 5.30). Two classes: one sigmoid node or two softmax nodes, the same model. Logistic regression is the network with $L = 1$; a classification network is logistic regression on learned features.

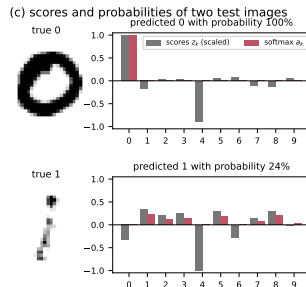
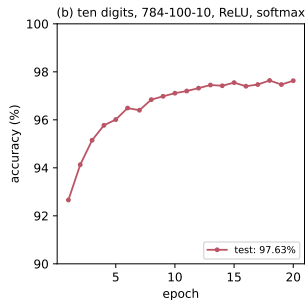
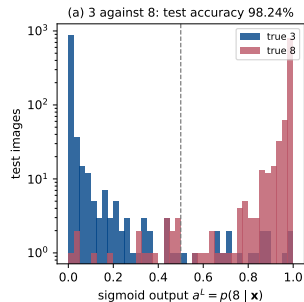
Why the pairing matters: a sigmoid output with the squared error (week41_delta_heads.pdf)



One sigmoid output node, target $y = 1$. **(a)** The cross entropy $-\ln \sigma(z)$ grows linearly for $z \rightarrow -\infty$, the squared error saturates at $\frac{1}{2}$. **(b)** The output error that starts the backward pass: $|a - y| \rightarrow 1$ for the cross entropy, the more wrong the stronger the signal; $|a - y| \sigma'(z)$ for the squared error, which *vanishes* where the network is confidently wrong – at $z = -5$ a factor 150 smaller. A saturated, wrong output node learns almost nothing from its mistake. For a real-valued target the linear output with the squared error (grey) has no saturation at all. The same mechanism, f' multiplying the error, is the vanishing-gradient problem of Section 8.9 one layer deep.

Example B: the sigmoid and the softmax head on MNIST

(week42_mnist.pdf)



The code of week 41, unchanged. **(a)** Binary: the 11982 threes and eights of the training set, a 784–32–1 ReLU network with the sigmoid output and the cross entropy, $\gamma = 0.1$, minibatches of 100, ten epochs, one second: test accuracy 98.24% on 1984 images (35 wrong). The output is a probability: 61 test images have $0.2 < p < 0.8$ – the network says it is unsure – and the threshold 0.5 is a decision we add afterwards. **(b)** Ten classes: the 784–100–10 network with the softmax and the multiclass cross entropy, twenty epochs, 97.63% as last week. **(c)** What the output layer actually produces: ten scores z_k (grey, scaled) and the softmax probabilities a_k (red). Top: a zero predicted with probability 1.0000 – one score far above the rest, the softmax turns a gap into a certainty. Bottom: the least confident of the 10 000 test images, a one predicted correctly with 24%, with 20% and 19% for the next two classes – three scores within a few tenths of each other. The softmax is shift-invariant: only the *differences* of the scores matter.

Pause and think 2: which of the two images trains the network?

Question

The backward pass starts with $\delta^L = a^L - y$, Eq. (8.69). Write it out for the two test images of Figure (c): the zero with $a_0 = 1.0000$, and the one with $a_1 = 0.24$, $a_k \approx 0.20$ and 0.19 for two other classes. Which image moves the weights, by how much, and in which direction? Why does the softmax need the cross entropy for this to work – what would the squared error $\frac{1}{2} \sum_k (a_k - y_k)^2$ have given for the confident *wrong* answers of last week's error gallery, the “97% zero” that was not a zero?

Pause and think 2: which of the two images trains the network?

Question

The backward pass starts with $\delta^L = a^L - y$, Eq. (8.69). Write it out for the two test images of Figure (c): the zero with $a_0 = 1.0000$, and the one with $a_1 = 0.24$, $a_k \approx 0.20$ and 0.19 for two other classes. Which image moves the weights, by how much, and in which direction? Why does the softmax need the cross entropy for this to work – what would the squared error $\frac{1}{2} \sum_k (a_k - y_k)^2$ have given for the confident *wrong* answers of last week's error gallery, the “97% zero” that was not a zero?

Answer

For the zero, $\delta_0 = 1.0000 - 1 \approx 0$ and every other $\delta_k = a_k \approx 0$: the sample contributes essentially nothing to $(A^{L-1})^T \Delta^L$ – a confident correct answer is *finished* and the gradient ignores it. For the one, $\delta_1 = 0.24 - 1 = -0.76$ and $\delta_k = +0.20, +0.19, \dots$ for the rivals: a strong push that raises the score of class 1 and lowers the others, and the K errors sum to zero (the shift that the softmax cannot see). The cross entropy makes the signal proportional to the *probability mass* on the wrong classes, however the scores are placed. With the squared error on the softmax, (8.65) keeps the Jacobian $a_k(\delta_{kj} - a_j)$ in the output error, and for a confident wrong answer – $a_{\text{wrong}} = 0.97$, $a_{\text{true}} = 0.03$ – every factor $a_k(1 - a_k)$ is small: the network would learn least from exactly the mistakes it is surest about. The pairing of (8.69) is not a convenience; it is what makes the confident mistakes the *loudest* ones.

A network with no data: the equation is the cost (Section 9.1)

The problem

An ordinary differential equation of order n , Eq. (9.1), $f(x, g(x), g'(x), \dots, g^{(n)}(x)) = 0$, with initial or boundary conditions that make the solution unique. No labels: nobody hands us $g(x_i)$. What we have is the equation.

The trial solution, Eq. (9.2)

$$g_t(x, P) = h_1(x) + h_2(x, N(x, P))$$

$N(x, P)$ is our network with parameters P ; h_1 alone satisfies the conditions; h_2 is built so that the network's contribution *vanishes* wherever a condition is imposed. Then the conditions hold *exactly, for every P* – at initialisation, after a bad step, always. The optimiser has one job: make the equation hold.

The cost, Eq. (9.3): the mean squared residual

At N collocation points x_i that we choose,

$$C(x, P) = \frac{1}{N} \sum_{i=1}^N \left(f(x_i, g_t(x_i), g'_t(x_i), \dots) \right)^2.$$

Three remarks. The x_i are not measurements: place them anywhere, change them at will. It is a mean squared error, so Chapter 4 applies unchanged – Adam of week 38, the residual in the role of the target. It is *not* convex, and we shall see a run converge confidently to the wrong answer.

What is new, and what is not

New: the residual needs g'_t , hence $\partial N / \partial x$ – a derivative with respect to the *input*, not the parameters. Not new: the network, its initialisation, the optimiser, the parameter gradient. The idea is Lagaris, Likas and Fotiadis (1998); as *physics-informed neural networks* it is a large research area today.

Differentiating the network with respect to its *input*: forward mode (9.2, Eqs. 9.4–9.6)

Differentiate the forward pass $z^l = a^{l-1}W^l + b^l$, $a^l = f(z^l)$ with respect to an input coordinate x_k , starting from $\partial a^0 / \partial x_k = e_k$:

$$\frac{\partial z^l}{\partial x_k} = \frac{\partial a^{l-1}}{\partial x_k} W^l, \quad \frac{\partial a^l}{\partial x_k} = f'(z^l) \circ \frac{\partial z^l}{\partial x_k} \quad (9.5)$$

and once more, with the product rule,

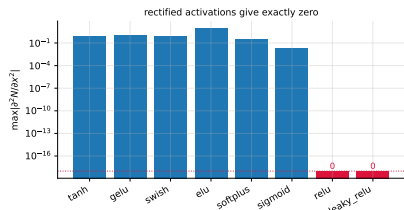
$$\frac{\partial^2 a^l}{\partial x_k^2} = f''(z^l) \circ \left(\frac{\partial z^l}{\partial x_k} \right)^2 + f'(z^l) \circ \frac{\partial^2 z^l}{\partial x_k^2} \quad (9.6)$$

The derivatives travel *forward* with the activations: one extra matrix product per layer per order, three forward passes for N, N', N'' . This is forward-mode AD (Section 4.14), and the right mode here: *one* input, many parameters – the reverse of training, where there is one cost and many parameters, which is why the parameter gradient still goes backwards.

```
def network_derivs(layers, X, activation="tanh", k=0):
    """N, dN/dx_k, d2N/dx_k^2 by the forward recursions (9.5), (9.6)."""
    f, fp = ACTIVATIONS[activation] # the week-41 table
    fpp = {"tanh": lambda z: -2.0*np.tanh(z)*(1.0 - np.tanh(z)**2),
           "sigmoid": lambda z: sigmoid_prime(z)*(1.0 - 2.0*sigmoid(z)),
           "relu": lambda z: np.zeros_like(z)}[activation]
    a = np.asarray(X, dtype=float)
    da = np.zeros_like(a) + (np.arange(a.shape[1]) == k) # da0/dxk = e_k
    d2a = np.zeros_like(a)
    for l, (W, b) in enumerate(layers):
        z, dz, d2z = a @ W + b, da @ W, d2a @ W
        if l < len(layers) - 1:
            a, da, d2a = f(z), fp(z)*dz, fpp(z)*dz**2 + fp(z)*d2z
        else: # linear output
            a, da, d2a = z, dz, d2z
    return a[:, 0], da[:, 0], d2a[:, 0]
```

Three lines per layer added to `feed_forward`, the activation table of week 41 reused. Checked against `jax.grad` of the same network: 2×10^{-16} in N' and N'' for `tanh`, 10^{-17} for the `sigmoid`, 0 for `ReLU` – that last zero is the next frame. The hand recursion is for understanding; for the PDEs we differentiate the *trial solution* itself by nested AD (next code frame).

The activation now matters more: f'' (Section 9.2, Figure 9.1)



Book Figure 9.1: $\max |d^2 N / dx^2|$ of a freshly initialised $[1, 20, 20, 1]$ network, by activation.

Our own run of the same experiment (week42.ipynb, seed 2026):

activation	$\max N'' $	activation	$\max N'' $
tanh	2.9	ELU	18.9
sigmoid	0.059	GELU	1.16
softplus	0.26	Swish	0.72
ReLU	0	leaky ReLU	0

Not small: identically zero

Equation (9.6) needs f'' . For the rectified family $f'' \equiv 0$ wherever it is defined and f' is piecewise constant, so every term of (9.6) vanishes: a ReLU network has an identically zero second derivative almost everywhere. It cannot represent the curvature a second-order equation is about – Poisson, diffusion, wave – and no width or training rescues it. The smooth activations differ among themselves by two orders of magnitude and all of them work. **Our choice:** tanh – smooth, bounded, $f' = 1 - f^2$ and $f'' = -2ff'$ in terms of the function value. GELU is the usual default of the PINN literature.

Pause and think 3: a ReLU network in a second-order equation: what does the residual report?

Question

Take the diffusion equation $\partial_t u = \partial_x^2 u$ and the physics-informed cost of Part 4, where the network itself is the solution, $u = N(x, t, P)$, and the equation residual is $L_{\text{PDE}} = \frac{1}{N} \sum_i (\partial_t N - \partial_x^2 N)^2$. With ReLU hidden units, what is $\partial_x^2 N$, what does L_{PDE} reduce to, how does Adam minimise it, and what does the trained network look like? Is the residual you print at the end a lie?

Pause and think 3: a ReLU network in a second-order equation: what does the residual report?

Question

Take the diffusion equation $\partial_t u = \partial_x^2 u$ and the physics-informed cost of Part 4, where the network itself is the solution, $u = N(x, t, P)$, and the equation residual is $L_{\text{PDE}} = \frac{1}{N} \sum_i (\partial_t N - \partial_x^2 N)^2$. With ReLU hidden units, what is $\partial_x^2 N$, what does L_{PDE} reduce to, how does Adam minimise it, and what does the trained network look like? Is the residual you print at the end a lie?

Answer

$\partial_x^2 N \equiv 0$ almost everywhere, so $L_{\text{PDE}} = \frac{1}{N} \sum_i (\partial_t N)^2$, and the optimiser makes it zero by making N *independent of t* : every weight that multiplies the t input is driven to zero. The equation residual is then exactly zero, on every collocation point, for a solution that does not diffuse at all – a “perfect” residual and a meaningless answer. The initial and boundary terms of the cost fight back (the solution must be $\sin \pi x$ at $t = 0$ and 0 at the walls, which a t -independent N cannot do with $\partial_x^2 N = 0$ either), so the run ends in a compromise that satisfies nothing well. The printed residual is not a lie, it is *true of the wrong object*: the second derivative the equation asks for was never in the model. Hence the rule of Section 9.3: always plot the residual *and* compare against something – here the exact solution, in general a classical method. A residual that falls suspiciously fast deserves suspicion.

The code: the week-41 network in jax.numpy, Adam, and solve_de

```
def network(layers, X, activation="tanh"):
    """The week-41 forward pass in jax.numpy, linear output -> N(x)."""
    f = JAX_ACTIVATIONS[activation]
    a = X
    for l, (W, b) in enumerate(layers):
        z = a @ W + b
        a = f(z) if l < len(layers) - 1 else z
    return a[:, 0]

def d_dxk(fun, k):
    """d fun(layers, X)/d x_k, row by row; nest it for d2/dx_k^2."""
    def wrapped(layers, X):
        def one_row(xk, row):
            return fun(layers, row.at[k].set(xk)[None, :])[0]
        return jax.vmap(jax.grad(one_row), in_axes=(0, 0))(X[:, k], X)
    return wrapped

def adam_minimise(cost_fn, params, n_iter=2000, gamma=1e-2, ...):
    """Adam, Eqs. (4.67)-(4.71), on any nested structure of arrays."""
    @jax.jit
    def step(params, m, v, it):
        c, g = jax.value_and_grad(cost_fn)(params) # reverse mode, by JAX
        ... # week 38's update, leaf by leaf
        return params, m, v, c
    ... # the loop over iterations

def solve_de(residual, sizes, X, activation="tanh", n_iter=2000, gamma=1e-2, rng=
    None):
    """Minimise the mean squared residual, Eq. (9.3)."""
    layers = to_jax(create_layers(sizes, activation, rng)) # week 41's
        initialisation
    cost_fn = lambda layers: jnp.mean(residual(layers, X)**2)
    return adam_minimise(cost_fn, layers, n_iter=n_iter, gamma=gamma)
```

What is reused

create_layers with its $1/M_{l-1}$ initialisation; the forward pass $a @ W + b$ with the linear output, the "mse" head; the Adam update of week 38 line for line. jax.value_and_grad replaces our hand-written backpropagation: the residual is an explicit function of the weights and we *could* derive its gradient as in Section 8.8, but it now contains the input-derivative recursion as well, and the derivation carries no insight. This is the situation automatic differentiation exists for.

What is new

Two functions per problem – the *trial solution* (the conditions) and the *residual* (the equation) – and d_dxk: the derivative of a function of X with respect to one input column, one row at a time (jax.grad of a scalar, vectorised with jax.vmap). Everything else is the same five lines for every equation of this lecture, the PDEs included. jax.jit compiles the step once; 3000 iterations take two seconds.

Exponential decay in code: a trial solution, a residual, one call

```
kappa, g0 = 2.0, 10.0
X = np.linspace(0, 1, 50)[: , None] # 50 collocation points

def trial_decay(layers, X): # Eq. (9.9)
    return g0 + X[: , 0] * network(layers, X, "tanh")

def residual_decay(layers, X): # Eq. (9.11)
    dg = d_dxk(trial_decay, 0)(layers, X) #  $g_t' = N + x \, dN/dx$ , by AD
    return dg + kappa * trial_decay(layers, X)

layers, history = solve_de(residual_decay, [1, 40, 40, 1],
                           X, "tanh",
                           n_iter=3000, gamma=2e-2,
                           rng=np.random.default_rng(1))

xx = np.linspace(0, 1, 401)[: , None] # evaluate anywhere
g = trial_decay(layers, jnp.asarray(xx))
print(np.abs(g - g0*np.exp(-kappa*xx[: , 0])).max()) # 3.1e-03
```

Read it as a recipe

1. Choose collocation points – any points, no measurements.
2. Write g_t so that the conditions hold for every P (here $g_0 + xN$).
3. Write the residual of the equation in terms of g_t and its derivatives, obtained by d_dxk .
4. Minimise the mean squared residual with the optimiser of week 38.
5. Evaluate g_t wherever you like – and *compare with something*.

Logistic growth: change two lines (the residual, g_0). Poisson: change the trial solution to $x(1-x)N$ and nest d_dxk twice. The diffusion equation: two input columns.

Exponential decay, Eqs. (9.7)–(9.11): the simplest test, and a cautionary one

The equation and the trial solution

$g'(x) = -\kappa g(x)$, $g(0) = g_0$; exact $g = g_0 e^{-\kappa x}$; $\kappa = 2$, $g_0 = 10$. One condition, so $h_1 = g_0$ and $h_2 = xN$:

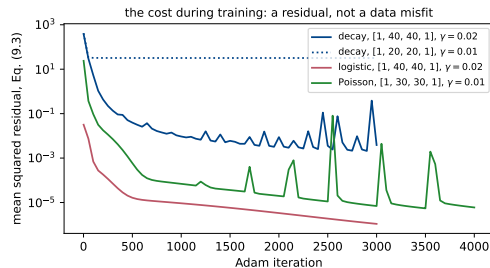
$$g_t(x, P) = g_0 + x N(x, P), \quad g'_t = N + x \frac{\partial N}{\partial x} \quad (9.9, 9.10),$$

$g_t(0) = g_0$ identically, because the factor x annihilates the network at the origin. Residual (9.11):

$$r = g'_t + \kappa g_t.$$

Result, $[1, 40, 40, 1]$, 50 points, 3000 Adam iterations at $\gamma = 0.02$

Cost (mean squared residual) 2×10^{-3} , maximum error 3.1×10^{-3} on $[0, 1]$ – relative error 3×10^{-4} of g_0 . The book's three seeds give 4.5×10^{-3} , 4.1×10^{-4} and 2.0×10^{-2} : accurate, and varying by a factor fifty between runs that differ only in the initialisation.



A cautionary result (Section 9.3)

Two hidden layers of *twenty* units at $\gamma = 0.01$: the cost falls to 31.9 and stays there – after 3000 iterations, after 10 000, at $\gamma = 0.02$ – and the solution is wrong by 1.83, eighteen per cent. Forty units, or $\gamma = 0.05$, escape at once. This is the non-convexity of Section 8.7 in the flesh: a local minimum where the gradient is *genuinely* zero. A classical solver converges or reports failure; a network can converge confidently to the wrong answer. *Always plot the residual, always compare against something.*

Logistic growth, Eqs. (9.12)–(9.14), against forward Euler

A non-linear equation

$g'(t) = \alpha g(t)(A - g(t))$, $g(0) = g_0$; exact
 $g = Ag_0 / (g_0 + (A - g_0)e^{-\alpha A t})$; $\alpha = 2$, $A = 1$,
 $g_0 = 1.2$ – the population starts above the carrying
capacity and decays towards it. Same condition, same
trial solution $g_t = g_0 + tN$; only the residual changes:
 $r = g'_t - \alpha g_t(A - g_t)$. Non-linearity costs nothing in
this framework.

The classical competitor

Forward Euler, $g_{i+1} = g_i + \Delta t \alpha g_i(A - g_i)$, error
 $\mathcal{O}(\Delta t)$:

method	max error on $[0, 1]$
Euler, 10 steps ($\Delta t = 0.1$)	9.9×10^{-3}
Euler, 20 steps	4.7×10^{-3}
Euler, 50 steps	1.8×10^{-3}
Euler, 100 steps	8.9×10^{-4}
network, 50 points	1.2×10^{-4}

Reading the table honestly

The Euler errors halve when the step halves – first order,
as it should be – and the network with fifty points is ten
times more accurate than Euler with fifty steps, and has no
systematic lag (Figure (b) of the next frame). That
comparison flatters the network. Euler took microseconds;
the network took 3000 Adam iterations. A second-order
Runge–Kutta scheme with fifty steps would beat the
network at a cost still negligible beside it. The network's
case lies elsewhere (Section 9.9, at the end of Part 3).

Our numbers

$[1, 40, 40, 1]$, 3000 iterations at $\gamma = 0.02$, seed 1: final
cost 1.1×10^{-6} , max error 1.15×10^{-4} – the book's
number to the digit, because the construction is
deterministic once the seed is.

The Poisson equation (9.15)–(9.19) against finite differences

The equation and the trial solution

$-g''(x) = f(x)$ on $(0, 1)$, $g(0) = g(1) = 0$, with $f = (3x + x^2)e^x$ and the exact $g = x(1 - x)e^x$. Two conditions:

$$g_t(x, P) = x(1 - x) N(x, P) \quad (9.18),$$

zero at both ends for every P , $h_1 \equiv 0$. By the product rule (9.19) $g_t'' = -2N + 2(1 - 2x)N' + x(1 - x)N''$: all of N, N', N'' appear, the first example that fails outright with ReLU. Residual $r = -g_t'' - f$, `d_dxk` nested twice.

The classical competitor, Eqs. (9.20)–(9.22)

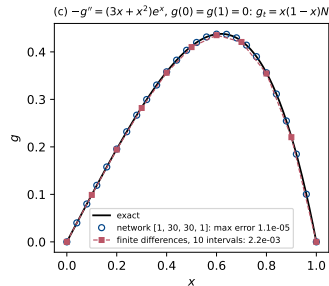
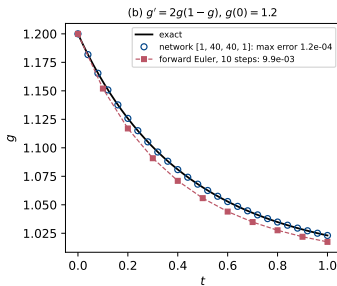
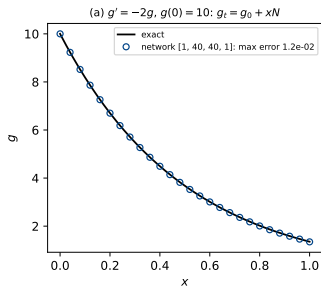
Taylor-expand $u(x_i \pm \Delta x)$ and add: the odd terms cancel, $u_i'' = (u_{i+1} - 2u_i + u_{i-1})/\Delta x^2 + \mathcal{O}(\Delta x^2)$, and the equation becomes a tridiagonal system with the discrete Laplacian, solved in $\mathcal{O}(n)$ operations (LU, Section 1.14).

method	max error
FD, 10 intervals ($\Delta x = 0.1$)	2.2×10^{-3}
FD, 20 intervals	5.4×10^{-4}
FD, 50 intervals	8.7×10^{-5}
FD, 100 intervals	2.2×10^{-5}
network [1, 30, 30, 1], 60 points, seed 1	1.2×10^{-5}
same, seed 2, 4000 iterations	2.1×10^{-2}
same, seed 2, 8000 iterations	5.7×10^{-6}

Reading it

The finite-difference error falls by four when Δx halves – $\mathcal{O}(\Delta x^2)$ confirmed – and the converged network sits at the hundred-interval level with sixty points: comparable accuracy per point, one tridiagonal solve against four thousand Adam iterations. The two seeds: same architecture, same γ , one run converged at 4000 iterations, the other sat at a cost of 0.027 until 8000 – and on a different machine (Apple silicon, another JAX version) seed 2 converges at 4000 while the decay error becomes 1.2×10^{-2} : rounding decides the path through a non-convex landscape. Here the classical method wins decisively; the example is a *verification*.

The three ordinary differential equations (week42_ode.pdf)



The network solution (circles, every sixteenth of 401 evaluation points) against the exact solution (line), with the classical competitor at its coarsest setting. **(a)** Decay: max error 3.1×10^{-3} on a solution of size 10. **(b)** Logistic growth: forward Euler with ten steps lags visibly behind the curve, the $\mathcal{O}(\Delta t)$ error of a first-order scheme; the network has no step and no lag. **(c)** Poisson: finite differences with ten intervals, error 2×10^{-3} , and the network at 10^{-5} . In every panel the collocation points were the 50 or 60 equally spaced points of the training, and the network is evaluated *between* them – it returns a function, not a table.

Pause and think 4: design the trial solution

Question

The trial solution carries the conditions, and the factor in front of N must vanish wherever a condition is imposed. Write g_t for (i) $g(0) = g_0$ and $g'(0) = v_0$ (a second-order initial-value problem); (ii) $g(0) = a$, $g(1) = b$ with $a, b \neq 0$; (iii) $g(0) = 0$ and $g'(1) = 0$ (a Neumann condition at the right end). Which of the three is awkward, and why?

Pause and think 4: design the trial solution

Question

The trial solution carries the conditions, and the factor in front of N must vanish wherever a condition is imposed. Write g_t for (i) $g(0) = g_0$ and $g'(0) = v_0$ (a second-order initial-value problem); (ii) $g(0) = a$, $g(1) = b$ with $a, b \neq 0$; (iii) $g(0) = 0$ and $g'(1) = 0$ (a Neumann condition at the right end). Which of the three is awkward, and why?

Answer

(i) $g_t = g_0 + v_0 x + x^2 N$: at $x = 0$ the value is g_0 ; the derivative $v_0 + 2xN + x^2 N'$ is v_0 at $x = 0$ because every term with N carries at least one factor x – the t^2 device of the wave equation, Eq. (9.33). (ii) $g_t = a + (b - a)x + x(1 - x)N$: h_1 is the straight line through the two boundary values, h_2 vanishes at both ends. (iii) is the awkward one: h_2 must vanish at $x = 0$ and have zero *derivative* at $x = 1$, and no prefactor alone does it – $x(2 - x)N$ has derivative $N'(1)$ at $x = 1$. One has to subtract what the network does at the boundary, e.g. $g_t = xN(x) - \frac{1}{2}x^2[N(1) + N'(1)]$, whose derivative at $x = 1$ is $N(1) + N'(1) - N(1) - N'(1) = 0$: doable here, a different formula for every geometry and every type of condition, and impossible on an L-shaped domain. That is the straitjacket Section 9.10 removes, at a price we shall measure.

Partial differential equations: the same strategy, two inputs (9.6)

Nothing changes but the input layer

$f(x_1, \dots, x_N, \partial g / \partial x_1, \dots, \partial^n g / \partial x_N^n) = 0$ (9.23), the trial solution $g_t = h_1(x) + h_2(x, N(x, P))$ (9.24), and the cost (9.25) the mean squared residual over M collocation points x_i forming the rows of X . The network now has N input nodes – `sizes = [2, 30, 30, 1]` for (x, t) – and that is the *only* change. Which is the point of having built it generally.

The derivatives

By nested AD on the trial solution, one coordinate at a time: `d_dxk(g, 1)` is $\partial_t g$, `d_dxk(d_dxk(g, 0), 0)` is $\partial_x^2 g$ – and the parameter gradient of the whole thing comes from `jax.value_and_grad`, as before.

The diffusion equation, Eqs. (9.26)–(9.28)

$$\frac{\partial g(x, t)}{\partial t} = \frac{\partial^2 g(x, t)}{\partial x^2},$$

with $g(0, t) = g(1, t) = 0$ and $g(x, 0) = u(x) = \sin \pi x$; exact solution $g = e^{-\pi^2 t} \sin \pi x$, which decays by $e^{-\pi^2} \approx 5 \times 10^{-5}$ over a unit of time.

Three conditions, one trial solution, Eq. (9.29)

$$g_t(x, t, P) = (1 - t) u(x) + x(1 - x) t N(x, t, P)$$

At $t = 0$ the second term vanishes and the first is $u(x)$; at $x = 0$ and $x = 1$ the factor $x(1 - x)$ kills the network and $u(0) = u(1) = 0$ kills the first term. All three hold identically, for every P .

The diffusion equation in code

```
def trial_diffusion(layers, X): # Eq. (9.29)
    x, t = X[:, 0], X[:, 1]
    return ((1 - t)*jnp.sin(jnp.pi*x)
            + x*(1 - x)*t*network(layers, X, "tanh"))

def residual_diffusion(layers, X): # Eq. (9.26)
    g_t = d_dxk(trial_diffusion, 1)
    g_xx = d_dxk(d_dxk(trial_diffusion, 0), 0)
    return g_t(layers, X) - g_xx(layers, X)

xs, ts = np.linspace(0, 1, 20), np.linspace(0, 1, 20)
Xg, Tg = np.meshgrid(xs, ts, indexing="ij")
X = np.column_stack([Xg.ravel(), Tg.ravel()]) # 400 points
layers, history = solve_de(residual_diffusion, [2, 30, 30, 1],
                           X,
                           "tanh", n_iter=800, gamma=1e-2,
                           rng=np.random.default_rng(1))
```

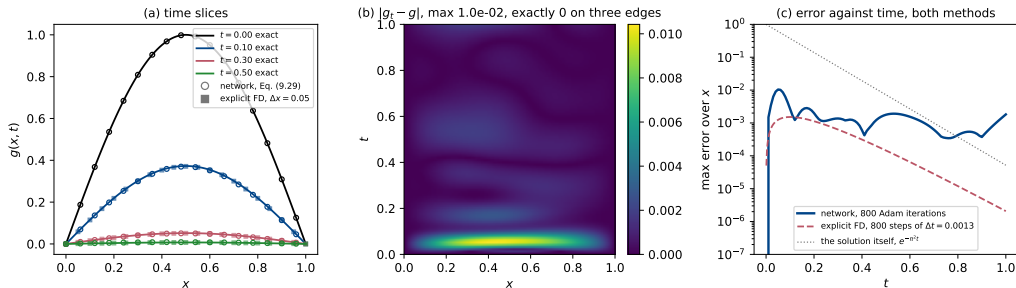
Count the changes

Two input columns; a trial solution with three conditions built in; a residual with one first and one second derivative. `solve_de`, `network`, `d_dxk` and `Adam` are untouched: 1021 parameters, 400 collocation points, 800 iterations, three seconds.

What to look at

Not the cost alone. Evaluate on a finer grid (101×101), compare with the exact solution, slice by time – and check the three edges where the conditions were imposed.

The diffusion equation solved (week42_diffusion.pdf)



$[2, 30, 30, 1]$, $\tanh$, 800 Adam iterations at $\gamma = 0.01$ (three seconds), evaluated on a 101×101 grid. **(a)** Time slices: the network (circles) on the exact curves; the squares are an explicit finite-difference scheme, $u_i^{n+1} = u_i^n + \alpha(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$ with $\Delta x = 0.05$ and $\alpha = \Delta t / \Delta x^2 = \frac{1}{2}$, the largest stable step – 800 steps of $\Delta t = 0.00125$ to reach $t = 1$. **(b)** The absolute error, largest (1.0×10^{-2}) at $x = 0.42$, $t = 0.05$ where the solution changes fastest, and *exactly* zero along $t = 0$, $x = 0$ and $x = 1$: not small, zero to machine precision, at every stage of training, because (9.29) satisfies the conditions identically. RMSE 1.8×10^{-3} ; 4000 iterations do not improve it (1.09×10^{-2} , 1.7×10^{-3}). **(c)** The maximum error over x against t : the network's stays near 10^{-3} while the solution (dotted) decays by four orders of magnitude; the finite-difference error decays *with* the solution. Max error 1.5×10^{-3} for the scheme, in milliseconds.

Pause and think 5: an absolute error that does not decay

Question

Panel (c): at $t = 1$ the exact solution is $e^{-\pi^2} \approx 5 \times 10^{-5}$ and the network's error is still about 10^{-3} – the *relative* error exceeds one, while the finite-difference solution tracks the decay. Why does the cost (9.25) not care? What does $\frac{1}{M} \sum_i r_i^2$ weight, and what would you change – in the collocation points, in the residual, or in the trial solution – to get the late times right?

Pause and think 5: an absolute error that does not decay

Question

Panel (c): at $t = 1$ the exact solution is $e^{-\pi^2} \approx 5 \times 10^{-5}$ and the network's error is still about 10^{-3} – the *relative* error exceeds one, while the finite-difference solution tracks the decay. Why does the cost (9.25) not care? What does $\frac{1}{M} \sum_i r_i^2$ weight, and what would you change – in the collocation points, in the residual, or in the trial solution – to get the late times right?

Answer

The cost is a *mean of squared absolute residuals* over collocation points spread uniformly over $[0, 1]^2$. Where the solution is 10^{-4} the residual of a network that returns 10^{-3} is 10^{-3} – invisible next to the residuals of 10^{-1} near $t = 0$ where the solution moves fast. Adam spends its effort where the squared residual is large, and that is early times. Remedies, in increasing order of knowledge used: (i) more collocation points at late times, or weights w_i on the residuals – cheap, and the beginning of the weighting problem of Part 4; (ii) a relative residual, $r_i/(|g_t| + \epsilon)$, or the equation for $\ln g$; (iii) build the decay into the trial solution, $g_t = e^{-\pi^2 t} [\sin \pi x + x(1-x)tN]$ – which is cheating here, since $e^{-\pi^2 t}$ is the answer, but is exactly what one does when the asymptotics are known and the shape is not. Information put into the ansatz is free and exact; information requested through the cost is paid for in optimisation effort – the theme of Part 4.

When is this worth doing? (Section 9.9)

The comparisons were deliberately unflattering

Runge–Kutta beats the network on the logistic equation, a tridiagonal solve on the Poisson equation, the explicit scheme on the diffusion equation – all of them by orders of magnitude in time. For textbook problems in one or two dimensions classical numerical analysis wins comfortably. Any presentation of this material that omits the comparison is selling something.

Machine-learning connection (Section 9.9)

The cost (9.25) is regression in disguise: the residual is the target, the collocation points are the data. Everything of Chapters 4 and 8 applies – and so do the difficulties: non-convexity (the 31.9 run), conditioning (Section 4.5), the activation deciding whether the derivatives exist, and no analogue of the theorem that says a scheme is $\mathcal{O}(\Delta x^2)$. We trade guarantees for flexibility.

Where the flexibility is needed

- ▶ **High dimension.** A mesh grows exponentially with the dimension; collocation points can be sampled, and the cost grows with the number of *parameters*. Hamilton–Jacobi–Bellman, the many-body Schrödinger equation: meshes are impossible, networks are not.
- ▶ **Complicated geometry.** No mesh to build; sample the domain.
- ▶ **The solution as a function.** Evaluate, differentiate, compose anywhere – no interpolation step. Both networks of this lecture were trained on 20×20 points and evaluated on 101×101 .
- ▶ **Inverse problems and data assimilation.** An unknown coefficient, a few measurements: add a data-misfit term and optimise over the coefficient alongside the weights. Unchanged machinery; for a classical solver an adjoint calculation and an outer loop. This is what “physics-informed” usually means – Part 4.

Physics-informed neural networks: drop the trial solution (9.10)

The straitjacket

h_1 and h_2 must be written by hand for every problem. For the unit box with homogeneous Dirichlet conditions $x(1-x)t$ does it; for a curved boundary, a Neumann or Robin condition, a coupled system, an L-shaped domain or an annulus, it ranges from tedious to impossible (Pause and think 4). Raissi, Perdikaris and Karniadakis (2019) coined the name; this section follows a tutorial written for the course by F. Nysæter, O. Fausko and K. Liodden.

The opposite view, Eq. (9.34)

Let the network itself be the solution, $u(x) \approx N(x, P)$, with no scaffolding – and *demand* the conditions of the optimiser instead, one least-squares penalty each. Hard constraints become *soft* ones, satisfied approximately at the optimum. Four point sets instead of one: N_{pde} collocation points in the interior, N_{ic} on $t = 0$, N_{bc} on each of $x = 0$ and $x = 1$.

The composite cost for the diffusion problem, Eqs. (9.35)–(9.38)

$$L_{\text{PDE}} = \frac{1}{N_{\text{pde}}} \sum_i (\partial_t N - \partial_x^2 N)_{(x_i, t_i)}^2,$$

$$L_{\text{IC}} = \frac{1}{N_{\text{ic}}} \sum_i (N(x_i, 0, P) - u(x_i))^2,$$

$$L_{\text{BC}} = \frac{1}{N_{\text{bc}}} \sum_i N(0, t_i, P)^2 + \frac{1}{N_{\text{bc}}} \sum_i N(1, t_i, P)^2,$$

$$L_{\text{total}} = L_{\text{PDE}} + \lambda_{\text{IC}} L_{\text{IC}} + \lambda_{\text{BC}} L_{\text{BC}}$$

L_{PDE} is exactly the cost we have minimised all along, with N in place of g_t . The weights λ play the role of the penalty parameters of Ridge regression (Section 3.8): how much we care about one group of residuals relative to another. Nothing in Chapters 4 and 8 changes.

The code: one point set becomes several, and the cost acquires weights

```
def pinn_solve(terms, sizes, activation="tanh", n_iter=2000, gamma=1e-2, rng=None,
...):
    """terms: list of (name, weight, residual_fn, points), Eq. (9.38)."""
    layers = to_jax(create_layers(sizes, activation, rng))
    def cost_fn(layers):
        return sum(*jnp.mean(residual(layers, Xk)**2) for _, w, residual, Xk in
            terms)
    return adam_minimise(cost_fn, layers, n_iter=n_iter, gamma=gamma, ...)

xs, ts = np.linspace(0, 1, 20), np.linspace(0, 1, 20)
Xi, Ti = np.meshgrid(xs[1:-1], ts[1:-1], indexing="ij") # interior only
X_col = np.column_stack([Xi.ravel(), Ti.ravel()]) # 324 points
X_ic = np.column_stack([xs, np.zeros(20)]) # t = 0
X_l = np.column_stack([np.zeros(20), ts]) # x = 0
X_r = np.column_stack([np.ones(20), ts]) # x = 1

def u_net(layers, X): return network(layers, X, "tanh") # Eq. (9.34)
u_t = d_dxk(u_net, 1)
u_xx = d_dxk(d_dxk(u_net, 0), 0)

def r_pde(layers, X): return u_t(layers, X) - u_xx(layers, X) # (9.35)
def r_ic(layers, X): return u_net(layers, X) - jnp.sin(jnp.pi*X[:, 0]) # (9.36)
def r_bc(layers, X): return u_net(layers, X) # (9.37)

terms = [("pde", 1.0, r_pde, X_col), ("ic", 10.0, r_ic, X_ic),
        ("bcL", 10.0, r_bc, X_l), ("bcR", 10.0, r_bc, X_r)]
layers, history = pinn_solve(terms, [2, 30, 30, 1], "tanh", n_iter=4000,
                             gamma=1e-2, rng=np.random.default_rng(1))
```

The whole difference, at the level of code

solve_de took one residual on one point set; pinn_solve takes a list of them with weights. The network, d_dxk, the initialisation and the optimiser are the ones of Part 2. A new condition is a new line in terms: a Neumann condition $\partial_x u = q$ is ("neu", 10.0, lambda L, X: u_x(L, X) - q, X_r), and an initial velocity for the wave equation is u_t on X_ic – the one condition that took thought in the trial solution takes one line here.

Why the interior points exclude the boundaries

A point carrying both the equation residual and a boundary residual is asked two things at once, and the two requests compete. 324 interior points, 20 on each edge.

Hard against soft on identical ground (Table 9.1)

Same [2, 30, 30, 1] network, same tanh, same Adam at $\gamma = 0.01$, same seed; errors on the 101×101 evaluation grid against (9.28).

Formulation	Iterations	max error	RMSE	error at $t = 0$
Hard, Eq. (9.29)	800	1.04×10^{-2}	1.79×10^{-3}	0 exactly
Hard, Eq. (9.29)	4000	1.09×10^{-2}	1.68×10^{-3}	0 exactly
Soft, $\lambda = 1$	800	2.93×10^{-2}	4.12×10^{-3}	2.93×10^{-2}
Soft, $\lambda = 10$	800	3.51×10^{-2}	1.35×10^{-2}	3.51×10^{-2}
Soft, $\lambda = 100$	800	4.00×10^{-2}	7.06×10^{-3}	1.48×10^{-2}
Soft, $\lambda = 10$	4000	3.18×10^{-2}	6.45×10^{-3}	6.29×10^{-3}

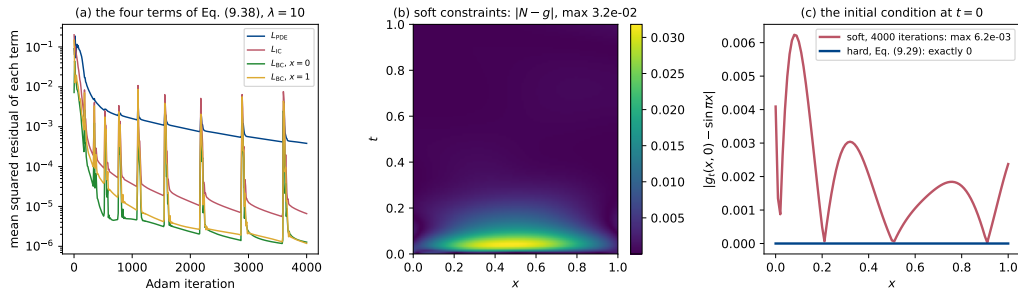
The hard formulation wins on every measure

RMSE three to four times smaller, and the error at $t = 0$ not merely smaller but zero. Five times more training does not close the gap: the soft solution's largest error is still 3×10^{-2} after 4000 iterations, at $x = 0.48$, $t = 0.04$, where the solution decays fastest. *On a problem where a trial solution can be constructed, constructing it is better.* Information built into the ansatz is free and exact; information supplied through the cost is paid for in optimisation effort and is never exact.

The weights are a real difficulty

λ from 1 to 10 changes little; 100 makes everything worse – the optimiser spends its capacity on the edges and L_{PDE} rises tenfold ($0.0009 \rightarrow 0.010$). There is a best λ , it is problem-dependent, and we found it by trying three values. The reason is conditioning (Section 4.5): the four terms have gradients of very different sizes – L_{PDE} carries W^2 through the second derivative, L_{IC} the value alone – and one γ must serve all. Choosing the weights automatically is an open research problem.

Soft constraints at work (week42_pinn.pdf)



$\lambda = 10$, 4000 iterations. **(a)** The four terms of Eq. (9.38), unweighted, during training: they descend at different rates and by different amounts, and the spikes every few hundred iterations are the terms trading against one another – an edge term jumps by two orders of magnitude when the equation term takes a step the edges do not like. **(b)** The error of the converged solution: largest in the fast-decaying region near $t = 0$, not on the boundaries, and *nowhere* zero. **(c)** The initial condition: the soft formulation misses $\sin \pi x$ by up to 6×10^{-3} after 4000 iterations; the hard formulation of Eq. (9.29) satisfies it identically – the blue line is not small, it is zero, and it would be zero for an untrained network.

So why soft constraints at all? The inverse problem, Eqs. (9.39)–(9.40)

An unknown coefficient and forty measurements

$\partial_t u = D \partial_x^2 u$ with D unknown, and instead of initial and boundary conditions a handful of noisy observations $\{(x_i, y_i)\}$ of the solution at scattered points. Minimise, over the weights *and* D together,

$$L(P, D) = \frac{1}{N_{\text{pde}}} \sum_i \left(\partial_t N - D \partial_x^2 N \right)_i^2 \\ + \lambda_{\text{data}} \frac{1}{N_{\text{data}}} \sum_i \left(N(x_i, P) - y_i \right)^2.$$

No initial condition, no boundary condition: the data replace them. A trial solution cannot even be written down – the soft formulation is not merely convenient, it is the only one available. For a classical solver this is an outer optimisation loop around the solver, with an adjoint equation derived by hand for the problem at hand.

```
rng = np.random.default_rng(3) # the "measurements"
X_obs = np.column_stack([rng.uniform(0, 1, 40), rng.uniform(0, 1,
40)])
y_obs = np.exp(-0.5*np.pi**2*X_obs[:, 1])*np.sin(np.pi*X_obs[:, 0])
\
+ rng.normal(0, 0.01, 40)

params = (to_jax(create_layers([2, 30, 30, 1], "tanh",
np.random.default_rng(1))),
jnp.array(D0)) # the weights, and D

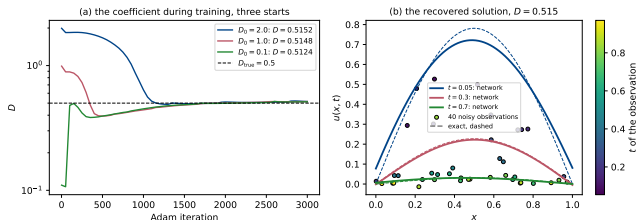
def cost_fn(params):
    layers, D = params # the only line that knows this is an inverse
                        problem
    return (jnp.mean((u_t(layers, X_col) - D*u_xx(layers, X_col))**2)
+ 10.0*jnp.mean((u_net(layers, X_obs) - y_obs)**2))

params, history = adam_minimise(cost_fn, params, n_iter=3000, gamma
=1e-2)
```

One more leaf of the parameter tree

Adam of week 38 does not care what a parameter *is*: D gets its own first and second moment and its own step, like every weight. Seven lines beyond the forward problem.

Recovering D from forty noisy points (week42_pinn_inverse.pdf)



$D_{\text{true}} = 0.5$, forty observations from the exact solution with Gaussian noise of standard deviation 0.01, seed 3; 324 interior collocation points. (a) Three starting values – 2.0, four times too large, 1.0 and 0.1 – converge to the same $D \approx 0.513$ within 1500 iterations. (b) The recovered solution (solid) against the exact one (dashed) at three times, with the observations coloured by the time they were taken; the visible gap at $t = 0.05$ is the three per cent error in D .

	D	rel. error
$D_0 = 2.0$	0.5133	2.7%
$D_0 = 1.0$	0.5148	3.0%
$D_0 = 0.1$	0.5124	2.5%
noise 0, $D_0 = 2$	0.4762	4.8%
noise 0.05, $D_0 = 2$	0.6903	38%

Not to be oversold

The first two noise rows differ by less than the seed-to-seed scatter and are the same result. The third is not: five per cent noise on the observations gives thirty-eight per cent error in D . Inverse problems are ill-posed; the sensitivity belongs to the problem, not to the method.

Pause and think 6: forty points, no conditions, three per cent – and then thirty-eight

Question

Equation (9.40) has no initial and no boundary term, yet the network recovered D to three per cent from forty noisy points and reproduced the solution in Figure (b). What supplied the missing conditions? Why does the recovered D degrade from 3% to 38% when the noise goes from 0.01 to 0.05 – is that the method's fault, and what should a report of " $D = 0.513$ " have come with?

Pause and think 6: forty points, no conditions, three per cent – and then thirty-eight

Question

Equation (9.40) has no initial and no boundary term, yet the network recovered D to three per cent from forty noisy points and reproduced the solution in Figure (b). What supplied the missing conditions? Why does the recovered D degrade from 3% to 38% when the noise goes from 0.01 to 0.05 – is that the method's fault, and what should a report of " $D = 0.513$ " have come with?

Answer

The forty observations, together with the equation. The equation propagates each measurement through space and time – a value at (x_i, t_i) constrains the solution everywhere the equation connects it to – and forty points scattered over the domain pin down a solution of the heat equation with a single free parameter. Conditions are information, and data are information; the cost only counts residuals. The collapse at 5% noise is *not* the method's fault: an inverse problem is ill-posed, the map from data to D amplifies noise, and a classical adjoint-based inversion on the same forty points would face the same amplification. Recovering a parameter from data is a *statistical estimation* problem, Chapter 2 applies, and a single number without an error estimate is not a result: bootstrap the observations (week 36), or vary the seed, and report $D = 0.51 \pm 0.02$ – or, at 5% noise, admit that forty points do not determine D to better than a factor of two.

Soft or hard? (Section 9.10)

Two points on a scale, not rivals

- ▶ If a trial solution can be constructed, construct it: the conditions hold exactly, the cost has one term, there are no weights to tune, and Table 9.1 shows the accuracy that buys.
- ▶ If the geometry is awkward, the conditions involve derivatives or mixed terms, or the equations are coupled, use soft constraints. The loss of accuracy is real but bounded; the alternative may be no method at all.
- ▶ If parameters of the equation are unknown and data are available, soft constraints are the only option, and extremely convenient.
- ▶ Mix them: a trial solution for the boundary conditions, a penalty for an initial condition – often the best of the three.

The wave equation, in one line (Sections 9.8, 9.10)

$\partial_t^2 g = c^2 \partial_x^2 g$ needs the initial velocity too. Hard: replace t by t^2 in the trial solution, Eq. (9.33) – a device that must be rethought if $v(x) \neq 0$. Soft: ("iv", 10.0, u_t, X_ic), one line. Accuracy after 4000 iterations: hard 4.8×10^{-4} , soft 8.7×10^{-3} , eighteen times apart, because the standing wave never decays and the boundary terms compete for the whole of training.

Trained on 20×20 , evaluated on 101×101

Both formulations return a differentiable function valid everywhere in the domain – not interpolation in the finite-difference sense, but evaluation of a closed form. It is one thing the method genuinely does better than a grid.

Where this is going: tomorrow, Project 2, next week

Tomorrow: one case, the backward pass

Your questions on today, then the structure of this week's exercises: the gradient of one layer by hand with every shape written down, two layers, the loop of Eqs. (8.65)–(8.68) for any depth, the batched version, each step checked against `jax.grad` – in `week42tuesday.ipynb` on a problem of our own, so that you do it once with us and once alone in `exercisesweek42.ipynb`. Deadline **Friday October 16**; declare your use of LLMs.

Next Monday, week 43

Convolutional neural networks (Chapter 10): keeping the spatial structure that we threw away when we flattened 28×28 into 784 – the reason the $4 \rightarrow 9$ and $7 \rightarrow 2$ confusions of week 41 survive a dense network and do not survive a convolutional one.

Project 2 (deadline November 6)

Your own `feed_forward`, `backpropagation` and `train` – exactly the code of the first half hour – on the Runge data and on the full MNIST set, against OLS/Ridge and logistic regression, with the optimisers of week 38, the resampling of week 36, a gradient check and a confusion matrix. Everything of Part 1 is a building block; the differential equations of Parts 2–4 are not part of the project, but the code is yours if you want a differential equation as your own data set.

Reading

Chapter 9, Sections 9.1–9.7 and 9.9–9.10; the exercises of Section 9.13 repeat Part 2 with GELU, Swish and the sigmoid, reproduce the local minimum, and add the wave equation with a non-zero initial velocity.

Summary of week 42

- ▶ **The reminder:** forward pass (8.21), the four equations (8.65)–(8.68) with their two transposes, the algorithm's five decisions; the hidden activation must be non-polynomial (Section 8.4 – a ReLU unit is a kink, and fifty kinks follow the Runge function to the noise floor) and it enters the backward pass only through f' ; the output activation follows the range of the target, the cost its distribution, and for linear + squared error, sigmoid + cross entropy, softmax + multiclass cross entropy the output error is $a^L - y$ – which makes the confident mistakes the loudest. On MNIST: a probability for 3-against-8, a distribution over ten classes.
- ▶ **Differential equations without data:** the trial solution $g_t = h_1 + h_2(x, N)$ carries the conditions exactly, the mean squared residual (9.3) is the cost, the derivatives with respect to the input come from the forward recursions (9.5)–(9.6) or nested AD, and ReLU has $f'' \equiv 0$ – hence tanh. Decay, logistic growth, Poisson: 10^{-3} , 10^{-4} , 10^{-5} , with one run stuck at a cost of 31.9 and one seed that needed twice the iterations. Euler and finite differences win on speed; the network returns a function.
- ▶ **The diffusion equation:** $g_t = (1 - t) \sin \pi x + x(1 - x)tN$, error exactly zero on three edges, 10^{-2} at $t = 0.05$, 10^{-3} absolute everywhere – a relative error above one at $t = 1$.
- ▶ **PINNs:** the conditions as soft penalties (9.35)–(9.38); hard beats soft on every measure when both are available, the weights λ are a conditioning problem; but with an unknown D and forty noisy points the soft formulation is the only one, and recovers D to three per cent – an estimate that needs an error bar.

Tomorrow and the deadline

Tuesday: your questions, then *one* case – the backward pass, step by step, with the shapes. Exercises week 42 (your own backpropagation against `jax.grad`, then training) are due **Friday October 16 at midnight**.